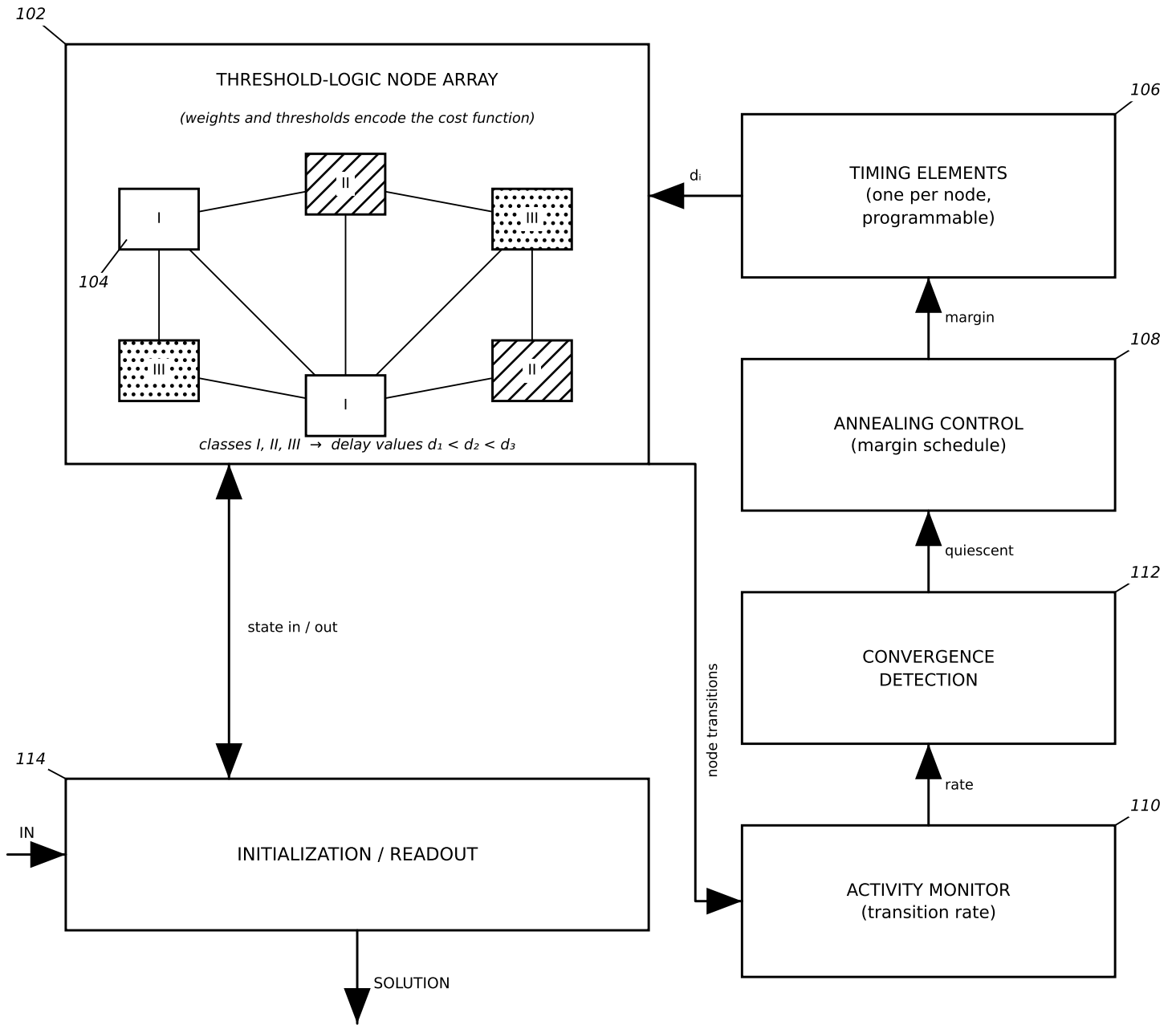


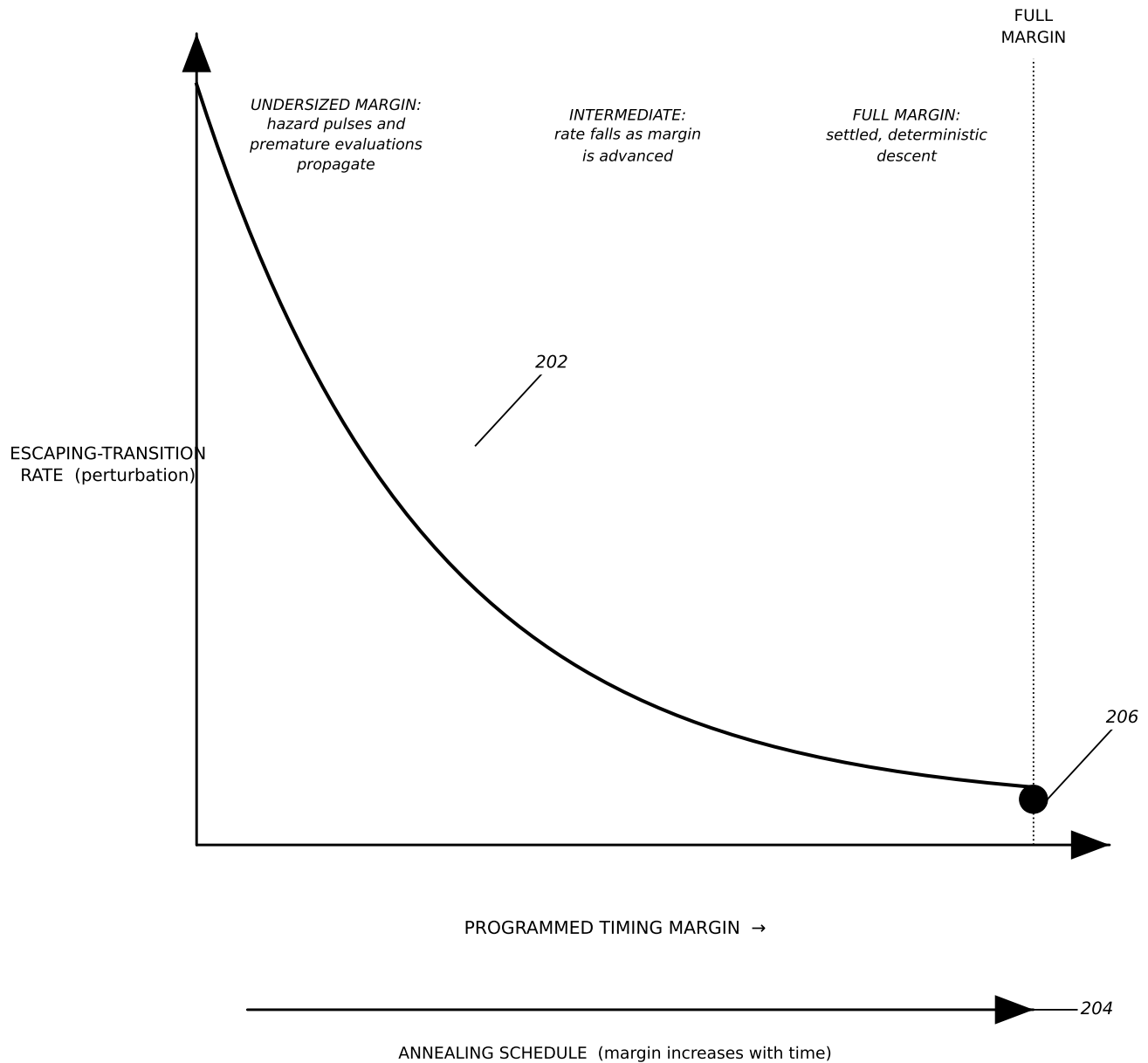
APPARATUS OVERVIEW — NO CLOCK SIGNAL IS PRESENT ANYWHERE IN THE APPARATUS



Every node evaluates continuously. Ordering is enforced only by the relative magnitudes of the per-node delay values — no clock, no sequencer, and no handshake between nodes of different classes.

FIG. 1

TIMING MARGIN AS THE ANNEALING CONTROL VARIABLE

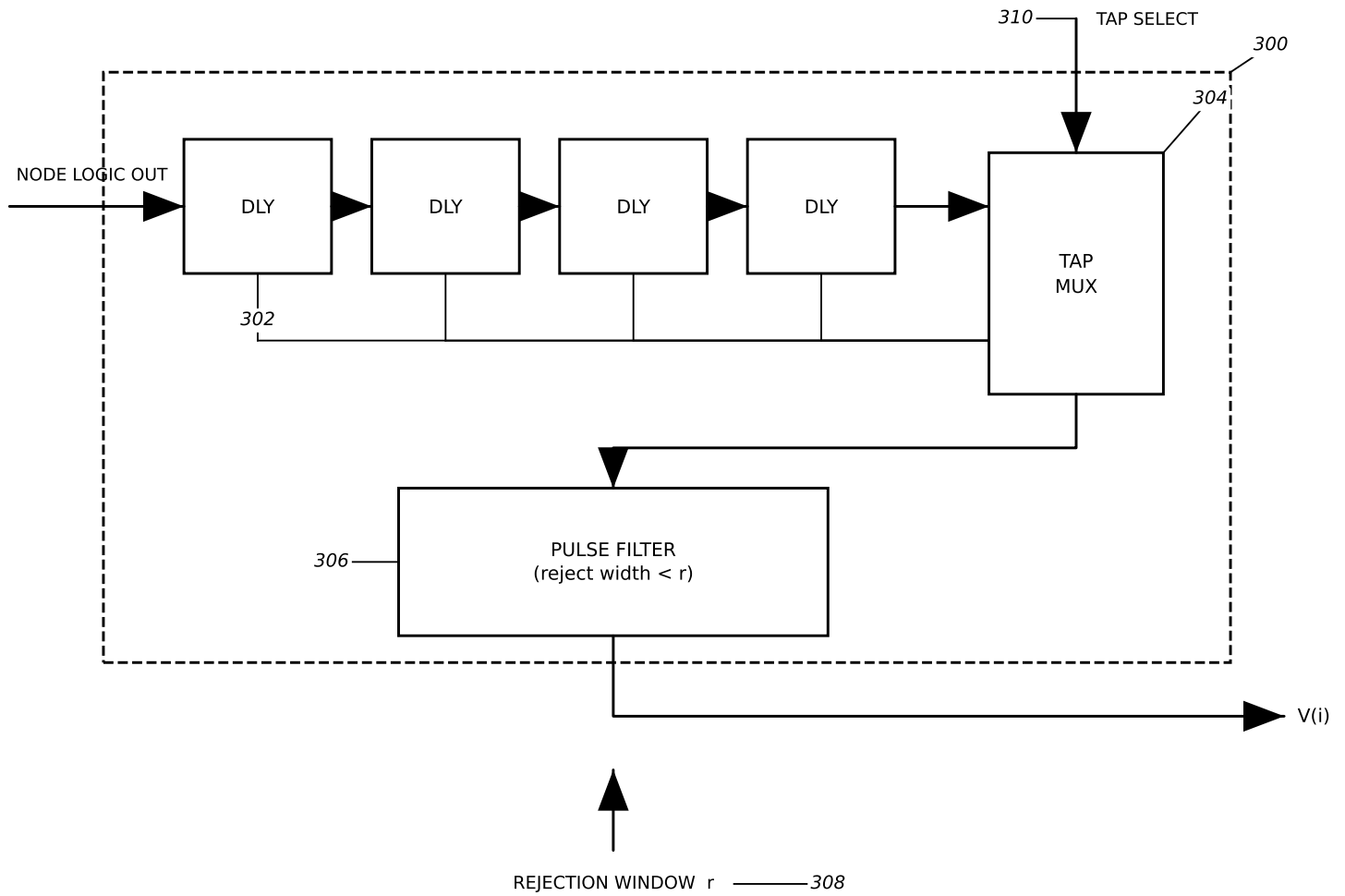


The perturbation rate is also proportional to the number of nodes still switching, so it falls to zero of its own accord as the network approaches a fixed point. The schedule therefore self-quenches and no separate cooling of a noise source is required.

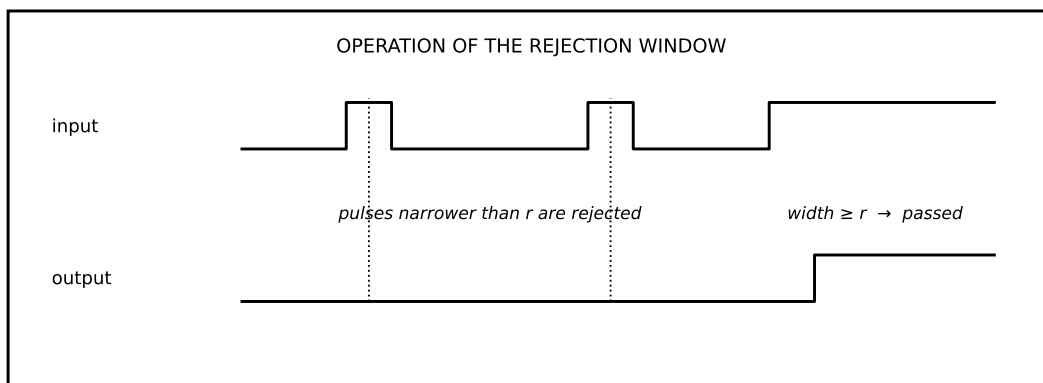
Relationship shown; axes are not to scale.

FIG. 2

PROGRAMMABLE INERTIAL DELAY ELEMENT



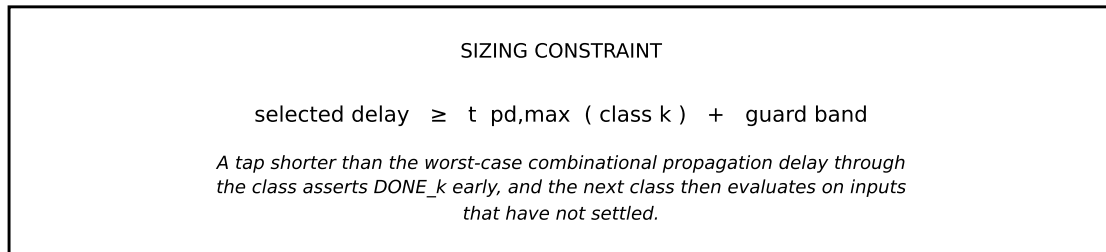
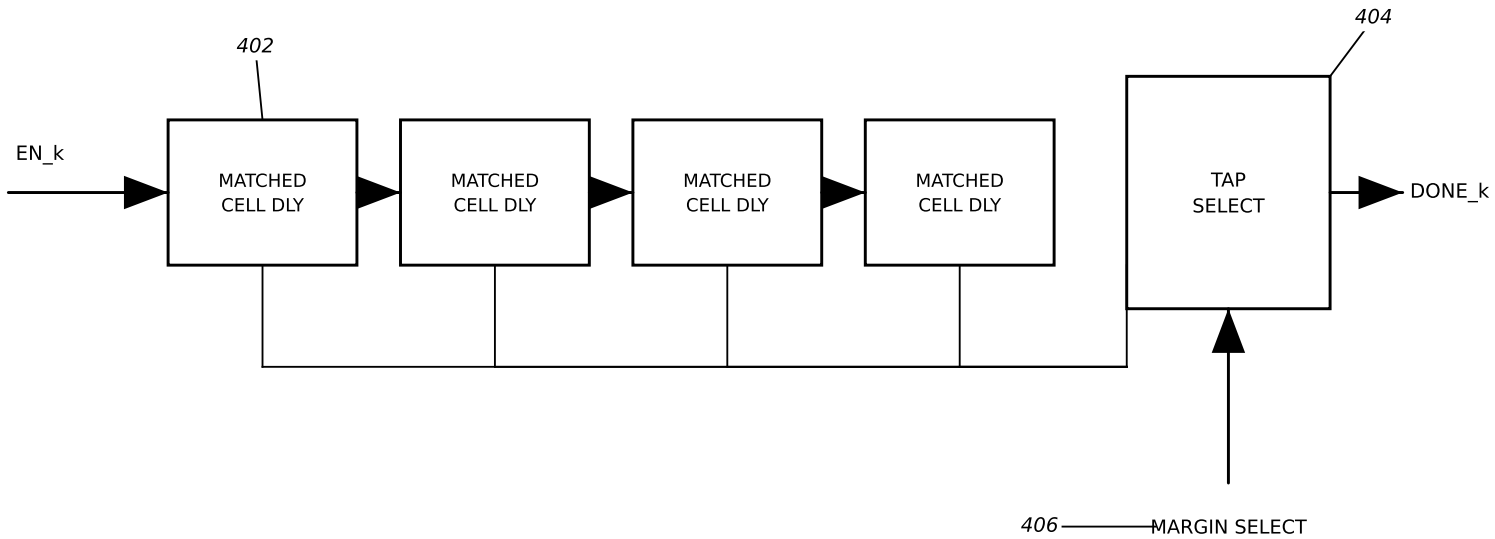
both controls driven by the annealing control (108)



A short rejection window lets hazard pulses through, and those pulses are the perturbation source. A full-width window admits at most one settled transition per evaluation. r is therefore the annealing control variable.

FIG. 3

PROGRAMMABLE MATCHED-DELAY COMPLETION GENERATOR

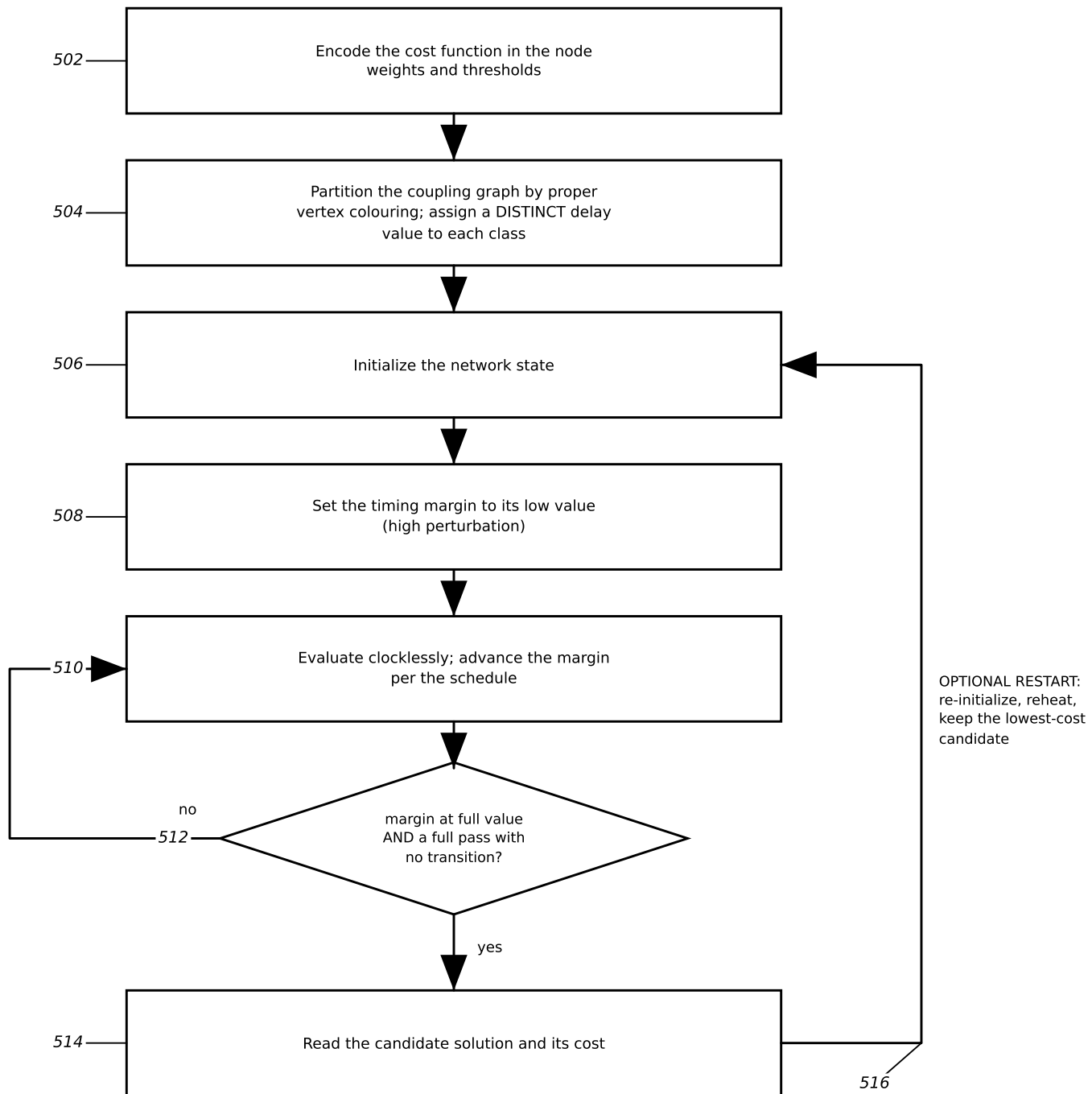


That premature evaluation is the perturbation the annealing schedule exploits: undersized taps early in the schedule, full-width taps at the end.

The same constraint applied to the node delays of FIG. 6 is a correctness requirement rather than a control variable — a node delay shorter than the propagation delay through its own logic makes the class ordering inoperative, because commits then interleave in an order the delay values no longer determine.

FIG. 4

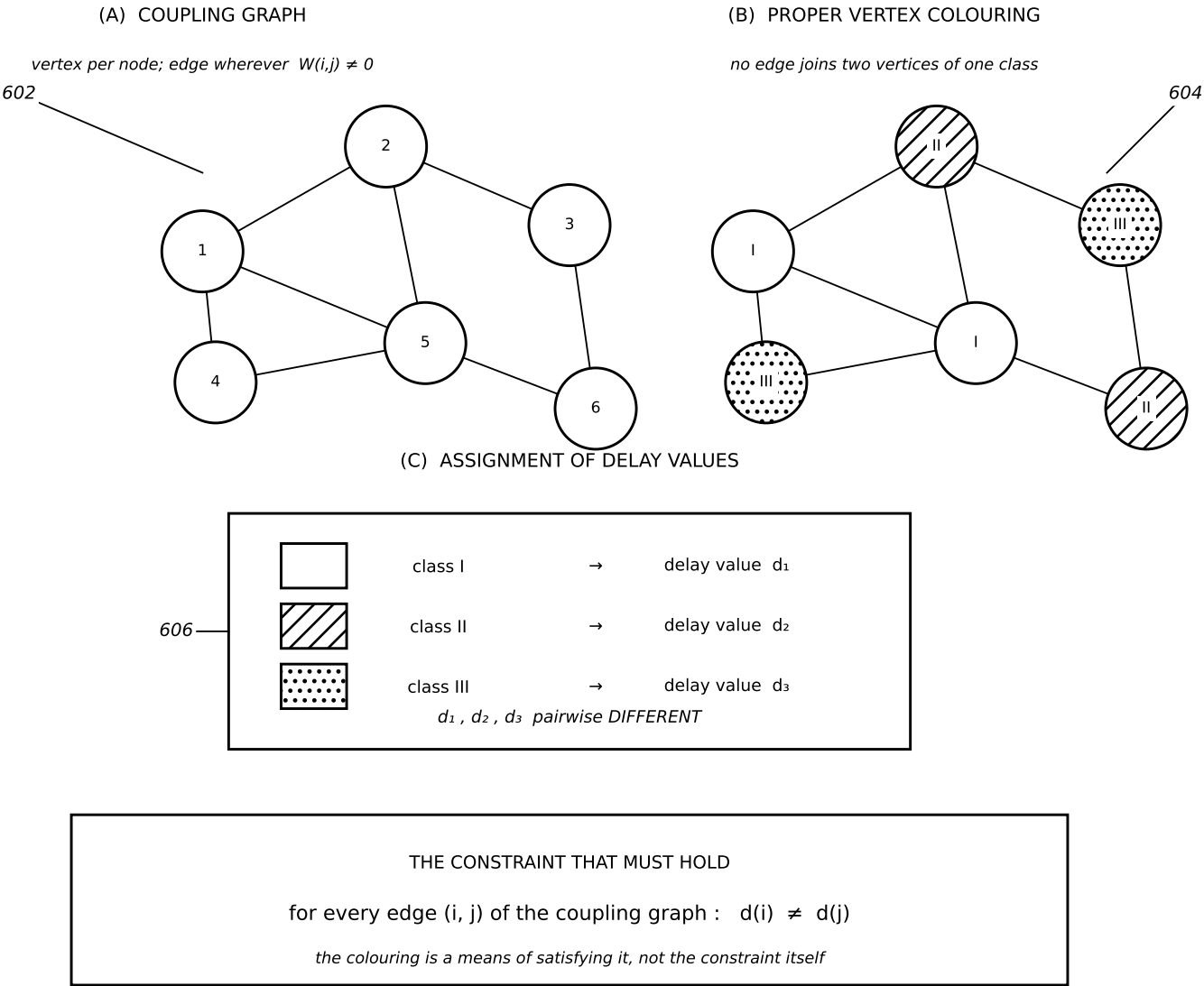
METHOD OF OPERATION



Steps 502 and 504 are performed once at compile time. Nothing inside the evaluation loop reprograms the node logic, so the loop rate is set by the settling time of the array and not by any reconfiguration time.

FIG. 5

DERIVATION OF THE NODE DELAY VALUES FROM THE COUPLING GRAPH

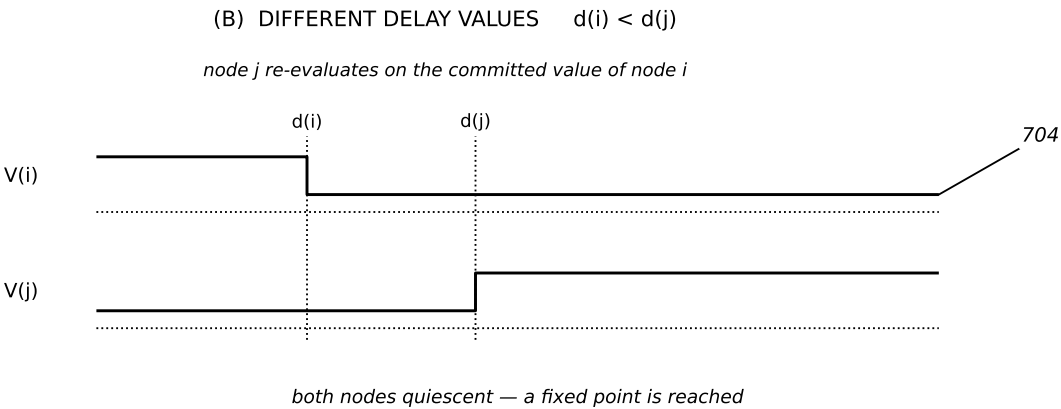
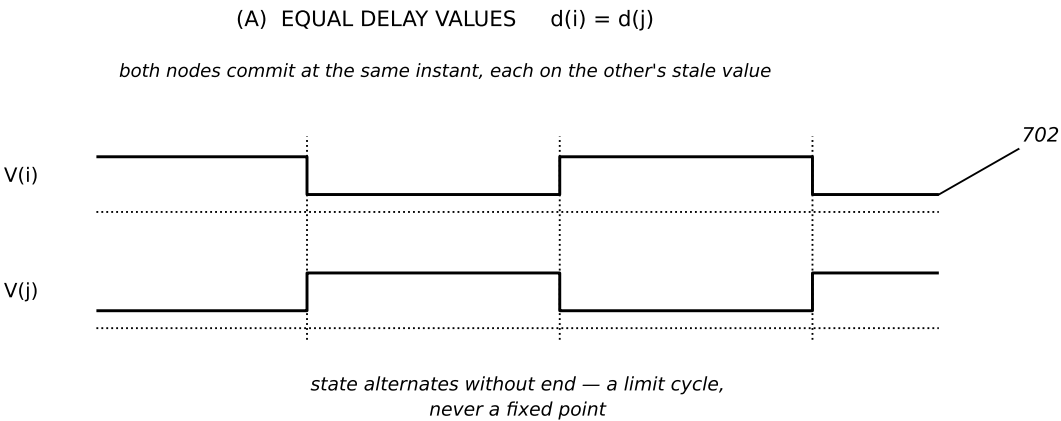


The constraint is on the DELAY VALUES, not on the class labels. A proper colouring whose classes are all assigned the same delay value satisfies every graph-theoretic requirement and does not work — see FIG. 7. The particular values are otherwise immaterial; only their distinctness on each edge is required.

Because the coupling graph is sparse, the number of distinct values needed is small and does not grow appreciably with the number of nodes.

FIG. 6

WHY THE DELAY VALUES MUST DIFFER — COUPLED PAIR (i, j)



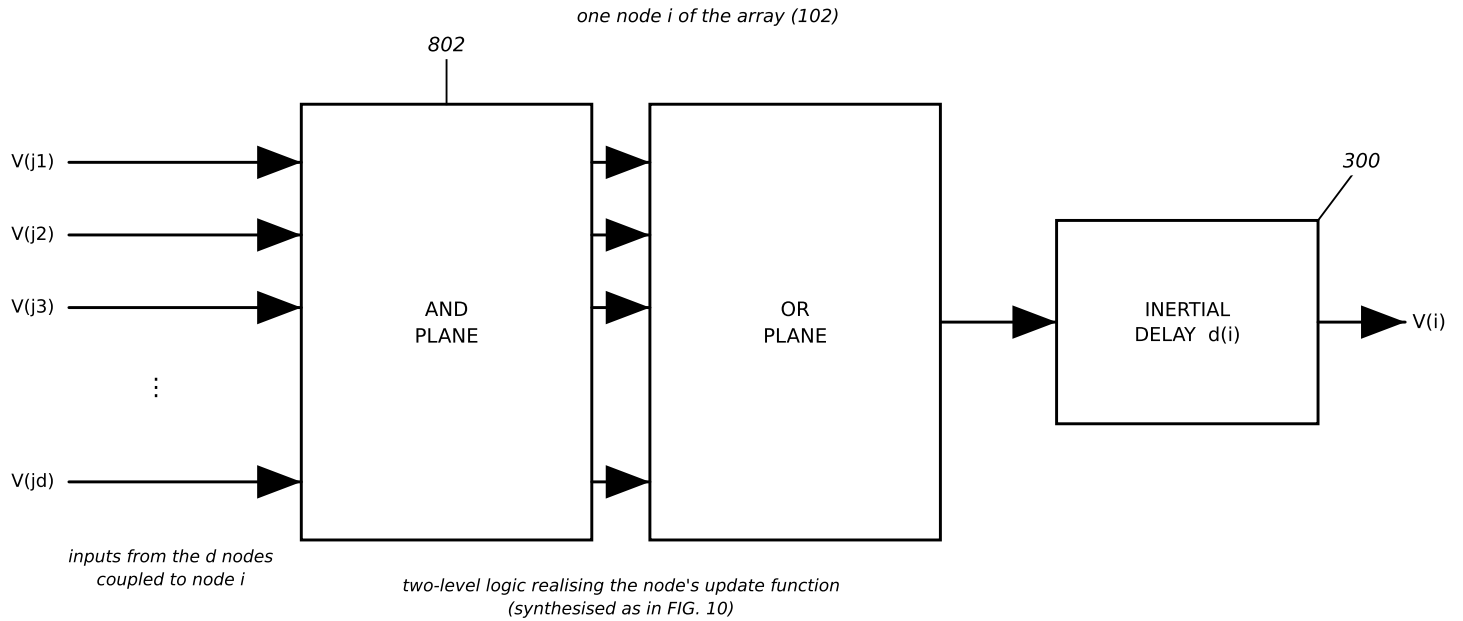
	MEASURED	
	distinct delay values	identical delay values
proper colouring	100 %	0 %
no colouring (parity)	0 %	—

fraction of initial states reaching a fixed point, N = 16, register-transfer simulation

*The upper-right cell is the reason the constraint is stated on delay values.
That configuration is a valid proper colouring of the coupling graph and it
settles in none of the trials.*

FIG. 7

NODE CIRCUIT AND THE DELAY SIZING RULE



$V(i)$ becomes an input to every node coupled to i , and to no other node. Node i does not receive its own output.

SIZING RULE FOR THE NODE DELAY

$$d(i) > t_{pd, \max}(\text{node } i) \quad \text{and} \quad d(i) \neq d(j) \text{ for every coupled } j$$

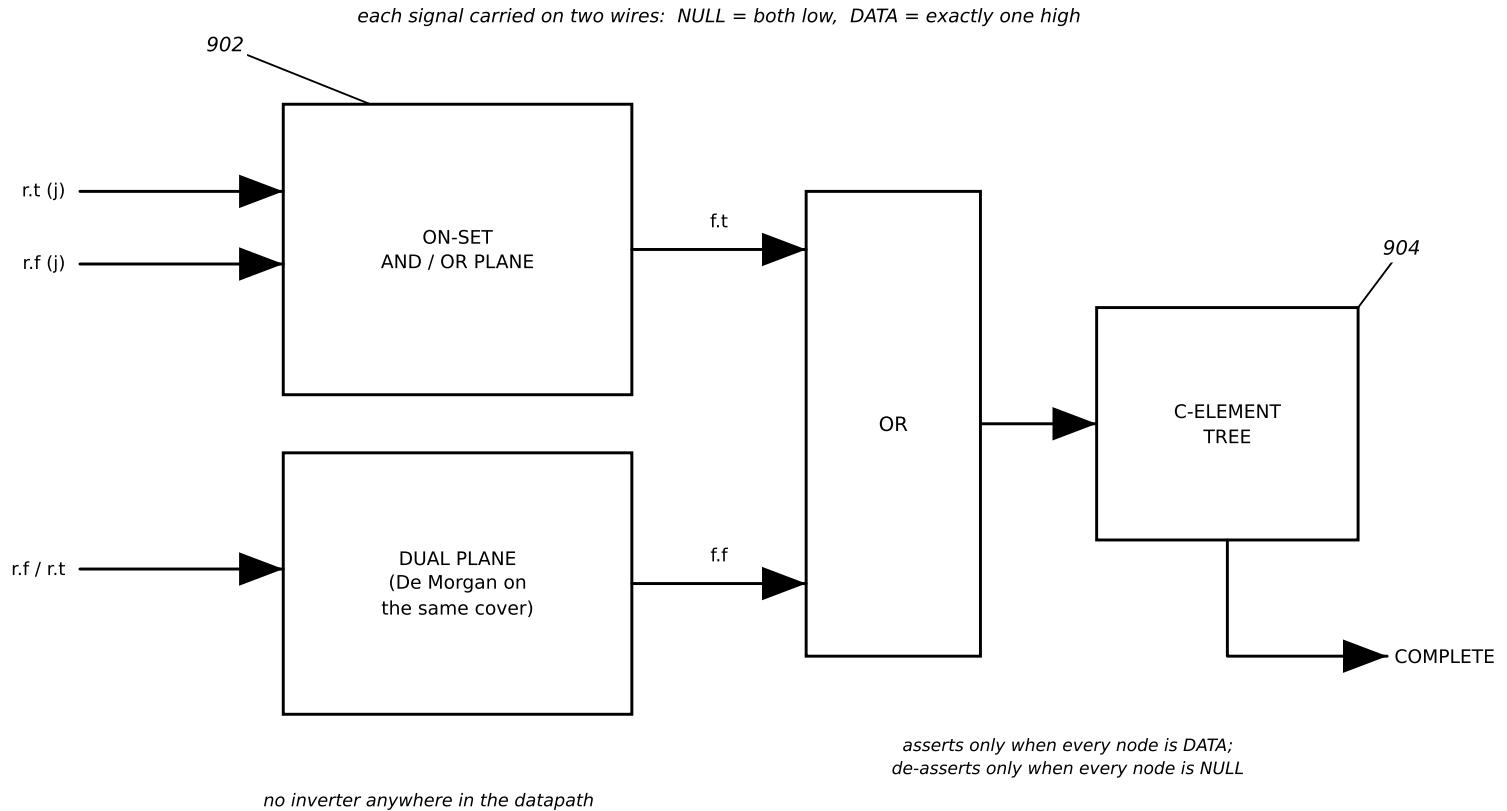
The first inequality is what makes the second one mean anything. If the node delay is shorter than the worst-case propagation delay through that node's own logic, commits interleave in an order the delay values no longer determine and the class ordering is inoperative — the array still runs, and still settles, but not under the schedule that was designed.

The two inequalities are independent. Satisfying only the second gives a circuit that violates its own schedule; satisfying only the first gives a correctly-timed circuit with no ordering at all.

The delay element is the same element as FIG. 3. When its rejection window is programmable, one element performs both functions: it sequences the node and it sets how much hazard activity is admitted.

FIG. 8

ALTERNATIVE EMBODIMENT — DELAY-INSENSITIVE NODE



WHY THIS IS HAZARD-FREE WITHOUT ANY TIMING ASSUMPTION

Out of NULL every rail rises and none falls, so every product term is a monotone function of the rails and every internal node can only go $0 \rightarrow 1$. A node that never falls cannot glitch. Correctness therefore does not depend on the sizing rule of FIG. 8.

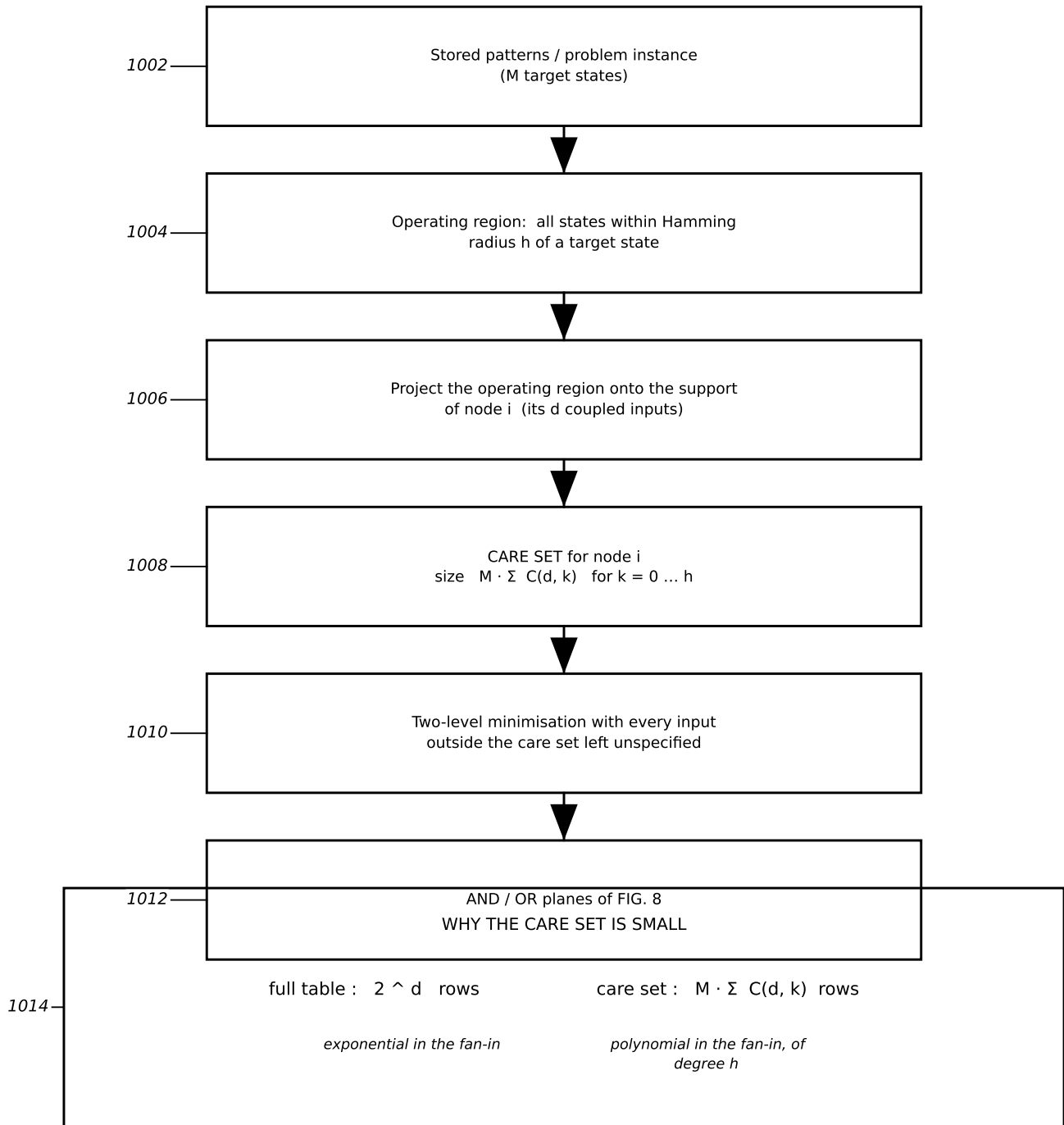
The false rail is obtained from the true rail's cover by De Morgan, not by a second independent minimisation. Two covers minimised against the same don't-care set leave inputs covered by neither, both rails then stay low, and completion detection waits for a DATA that never arrives.

This embodiment does NOT replace the partitioning. A C-element prevents a node from acting on inputs that have not settled; it does nothing about two coupled nodes acting on the same stale state. The same dual-rail array with the partitioning removed does not converge.

It costs roughly twice the logic of FIG. 8 and one NULL phase per evaluation.

FIG. 9

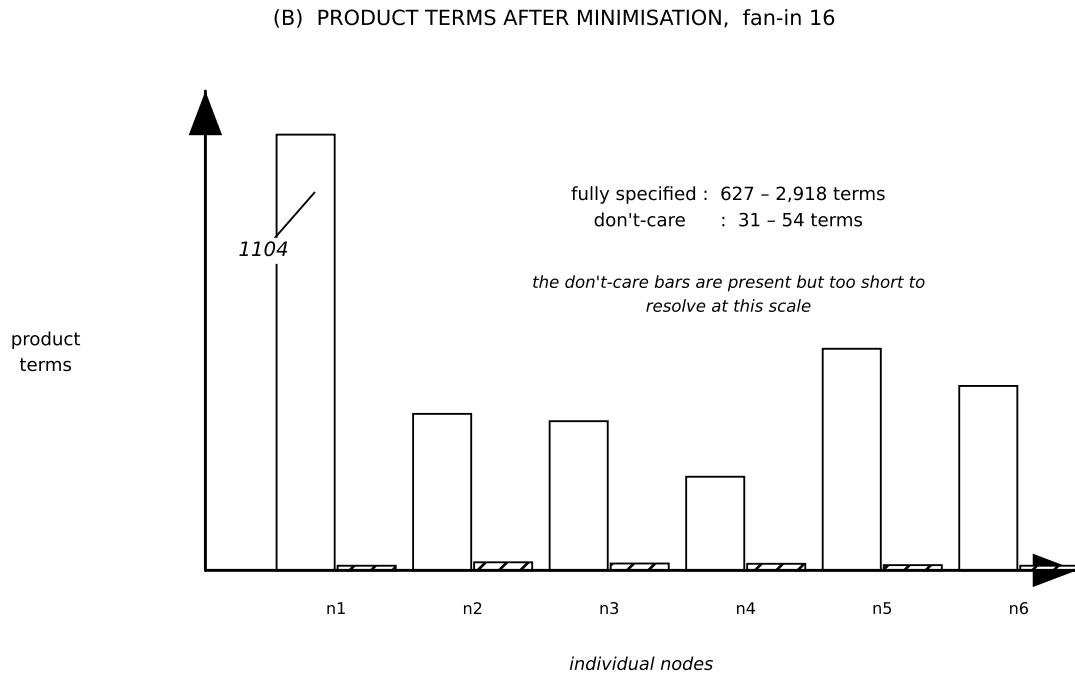
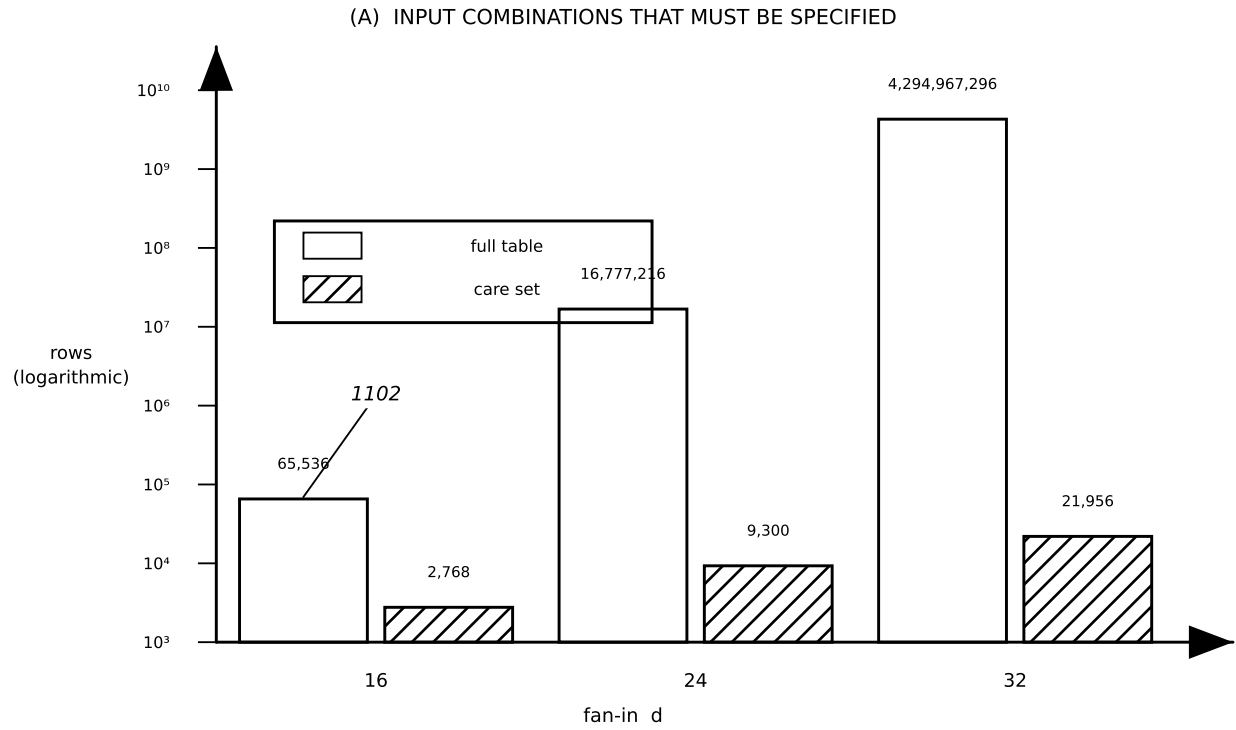
SYNTHESIS OF THE NODE LOGIC USING OPERATING-REGION DON'T-CARES



The care set is derived in closed form from the operating region. It is not obtained by sampling which inputs a trained network happens to visit, and it carries a guarantee over a stated radius rather than an observed frequency. Because the network is recurrent, an unspecified input that is later realised may create a fixed point that the target function does not have — a failure mode with no counterpart in a feed-forward network.

FIG. 10

MEASURED — CARE SET AND PRODUCT-TERM COUNT VERSUS FAN-IN



Conditions: 256 nodes, $M = 4$ target states, radius $h = 3$, two-level minimisation by the Berkeley espresso program. At fan-in 24 and above the fully-specified table cannot be enumerated at all, so no comparison column exists there.

The saving is in what has to be SPECIFIED, and only then in the term count that follows from it.

FIG. 11